Android 13 SDK
开发指南

版本号: 1.1

发布日期: 2025.11.03

广州星翼电子科技有限公司zdyzlinux

版本历史

版本号	日期	制/修订人	内容描述
1.0	2025.09.22	AWA1739	初始版本
1.1	2025.11.03	AWA1739	增加 ubuntu 安装环境说明

广州星翼电子科技有限公司zdyzlinux

版权所有 © 珠海全志科技股份有限公司。保留一切权利

目　录

1 前言 **1**
1.1 文档简介 1
1.2 目标读者 1
1.3 适用范围 1
1.4 名词解释 1

2 Android13 SDK 支持芯片 **2**

3 Android13 SDK 代码编译 **3**
3.1 代码编译 3
3.1.1 安装必要的软件 3
3.1.2 设置环境变量 3
3.1.3 lunch 介绍 3
3.1.4 编译单个模块 6
3.1.5 一键编译命令 7
3.1.6 安全固件配置 7
3.1.7 方案编译例子 8
3.1.7.1 T736 编译 Android 固件 longan 配置 8
3.1.7.2 T736 编译 Android 固件 android 配置 8
3.1.7.3 生成 debug 固件 9
3.1.8 无源码打包 9
3.1.8.1 工作原理 9
3.1.8.2 使用 9
3.1.8.3 编译打包 10
3.1.8.4 举例 10
3.1.9 PhoenixSuit 烧写固件 10
3.2 发布 11
3.2.1 发布固件流程 11
3.2.2 OTA 包 11
3.3 GKI 12
3.4 fastboot 烧写 12
3.4.1 fastboot 常用命令 13
3.4.2 设备解锁 13

4 方案定制 **14**
4.1 新增方案配置 14
4.1.1 新增 android 方案 14
4.1.2 新增 bsp 板级方案 14
4.2 配置分区 15

版权所有 © 珠海全志科技股份有限公司。保留一切权利

4.3 修改启动 LOGO . . . 15
4.4 修改开机动画 . . . 15
4.4.1 文件结构 . . . 16
4.4.2 desc.txt 配置文件 . . . 17
4.5 修改开机音乐 . . . 17
4.5.1 优化 png 图片 . . . 17
4.5.2 打包 bootanimation.zip . . . 17
4.6 修改充电图标 . . . 18
4.7 定制 recovery 功能 . . . 18
4.7.1 按键选择 . . . 18
4.8 方案文件替换 . . . 19
4.9 overlay 说明 . . . 20
4.9.1 为产品添加 Overlay 目录 . . . 20
4.9.2 改变 mk 文件来添加 overlays 的编译项 . . . 20
4.10 预装 APK . . . 21
4.10.1 预装到 system/app 目录 . . . 21
4.10.2 预装到 system/preinstall 目录 . . . 21

5 模块配置 22
5.1 自定义按键配置 . . . 22
5.1.1 Android 按键功能的映射 . . . 22
5.2 LCD Panel 配置 . . . 22
5.3 Touch Panel 配置 . . . 22
5.3.1 配置文件的修改 . . . 22
5.3.2 Android 层的配置修改 . . . 23
5.4 G-Sensor 配置 . . . 23
5.4.1 Android 层配置修改 . . . 23
5.4.1.1 首先调试 Z 轴 . . . 24
5.4.1.2 其次查看 X，Y 轴是否互换 . . . 24
5.4.1.3 再次调试 X，Y 轴方向 . . . 24
5.5 显示配置 . . . 25
5.6 WiFi/BT 配置 . . . 25
5.7 Camera 配置 . . . 25
5.7.1 Camera 驱动加载顺序 . . . 25
5.7.2 Camera 参数配置 . . . 26
5.7.3 配置文件的修改 . . . 26

6 Android 系统配置 27
6.1 壁纸设置 . . . 27
6.1.1 默认壁纸 . . . 27
6.1.2 添加壁纸 . . . 27
6.2 Launcher 默认图标和快捷栏设置 . . . 27
6.3 后台服务管理配置 . . . 28

版权所有 © 珠海全志科技股份有限公司。保留一切权利

6.3.1 功能介绍 28
6.3.2 方案配置 28
6.3.3 用户设置 29
6.3.4 功能调试 29

7 AIOT 方案配置 31
7.1 多屏方案配置 31
7.2 双触摸配置 31
7.3 Camera 配置 31

8 AAOS 系统配置 32
8.1 EVS 配置 32
8.2 音频配置 32

9 工具使用 33
9.1 DragonRunin 33
9.1.1 自动化 33
9.1.2 手动执行 34
9.1.3 测试结果 34
9.2 Keeptesting 36
9.3 DragonAtt 37

10 系统调试 38
10.1 Awlog 日志 38
10.2 adb 工具 38
10.2.1 常用命令 38
10.3 frame_monitor 工具 38
10.3.1 常用命令 39
10.4 cpu_monitor 工具 39
10.5 mtop 工具 40
10.5.1 常用命令 40
10.6 dumpsys 功能 41
10.6.1 常用命令 41
10.6.2 进阶 41
10.7 crashdump 模式 41

11 FAQ 43
11.1 Android13 内核编译环境依赖于 libelf-dev 库，编译之前请务必安装 libelf-dev 库。 43
11.2 Android13 编译过程中若突然退出，未编译完成再次编译，可能会出现无法继续编译的问题。 . . 43

广州星翼电子科技有限公司zdyzlinux

插　图

图 5-1　gsensor 23
图 9-1　测试 Pass 35
图 9-2　测试 Fail 36
图 10-1　frame_monitor 39
图 10-2　cpu_monitor 40
图 10-3　mtop 40

广州星翼电子科技有限公司zdyzlinux

版权所有 © 珠海全志科技股份有限公司。保留一切权利
文档问题反馈: aw-document@allwinnertech.com

表　格

表 1-1　术语表 1
表 2-1　Android13 SDK 支持芯片 2
表 3-1　构建参数表 4
表 6-1　Launcher 快捷栏配置表 28
表 9-1　Keeptesting 测试项 33
表 9-2　Keeptesting 测试项 37
表 9-3　DragonAtt 测试项 37
表 10-1　adb 常用命令 38
表 10-2　frame_monitor 常用命令 39
表 10-3　mtop 常用命令 40
表 10-4　dumpsys 基础命令 41
表 10-5　dumpsys 进阶命令 41

版权所有 © 珠海全志科技股份有限公司。保留一切权利

1 前言

1.1 文档简介

本文档介绍 Android 13 系统常见的定制开发问题，以帮助客户快速熟悉开发环境，实现快速方案移植。

1.2 目标读者

SDK 平台负责人、版本集成人员、SDK 开发人员、模块开发人员。

1.3 适用范围

Allwinner Android 13 平台。

1.4 名词解释

key	value
vendor-name	softwinner
platform-name	芯片平台名称
device-name	设备名称
product-name	产品名称
IC	芯片名称
BOARD	板卡名称

表 1-1: 术语表

版权所有 © 珠海全志科技股份有限公司。保留一切权利

2 Android13 SDK 支持芯片

芯片	Android 版本	内核版本	是否发布
A537	Android13	5.15	√
T736	Android13	5.15	√
A733	Android13	5.15	√
T527	Android13	5.15	√
T527V	Android13	5.15	√
A523	Android13	5.15	√
A527	Android13	5.15	√

表 2-1: Android13 SDK 支持芯片

广州星翼电子科技有限公司zdyzlinux

版权所有 © 珠海全志科技股份有限公司。保留一切权利

3 Android13 SDK 代码编译

3.1 代码编译

3.1.1 安装必要的软件

通常的，需要安装如下软件以支持 Allwinner BSP 编译及 Android 系统的编译。

每个 ubuntu 版本安装的的库是有所不同的，如下所示：

- ubuntu18.04

```
sudo apt-get install -y git-core gnupg flex bison gperf build-essential zip curl zlib1g-dev
    gcc-multilib g++-multilib libc6-dev-i386 lib32ncurses5-dev x11proto-core-dev libx11-dev
    lib32z-dev ccache libgl1-mesa-dev libxml2-utils xsltproc unzip elfutils-libelf-devel
    libelf-dev gawk fontconfig libncurses5
```

- ubuntu20.04

```
sudo apt-get install -y git gnupg flex bison gperf build-essential zip curl zlib1g-dev gcc-
    multilib g++-multilib libc6-dev-i386 lib32ncurses-dev x11proto-core-dev libx11-dev lib32z1
    -dev ccache libgl1-mesa-dev libxml2-utils xsltproc unzip libelf-dev gawk fontconfig python
     openssl libssl-dev libncurses5
```

注意

make3.8.2 存在 Bug, 在编译 Allwinner Linux-5.15 的内核时会导致编译报错，请升级到更高版本，或者降级到 3.8.1。

3.1.2 设置环境变量

build/envsetup.sh 文件中定义了若干命令，导入后才能够支持各种编译操作。

```
source build/envsetup.sh
```

3.1.3 lunch 介绍

在构建 Android 之前，必须先确定编译的目标。例如：

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
lunch t736_demo_car_arm64-userdebug
```

lunch 后接的字符串的组成部分包括：

- product_name 是期望编译的产品的名称，例如 t736_demo_car。可以根据设备的格式来命名特定的 product_name。
- 字符串的 build_variant 部分可以是下表中的三个值之一：

build_variant	说明
user	提供有限的安全访问权限，适用于生产环境。
userdebug	已启用 root 权限，其它与 user 基本一致。
eng	具有额外调试工具的开发配置。

表 3-1: 构建参数表

其它 launch 选项说明:

product_name	PMU	WIFI	RAM	LCD	CPU	Board
ceres_b3	AXP707	XR829	2GB	10’ 720P	A133	b3
ceres_p25t_arm64	AXP717	AW869	3GB	10’ 720P	A133P	p25t
ceres_p25_arm_go	AXP717	AW869	2GB	10’ 720P	A133	p25
ceres_p25_arm	AXP717	AW869	3GB	10’ 720P	A133	p25
ceres_p25_arm64	AXP717	AW869	3GB	10’ 720P	A133	p25
a523_pro_arm64	AXP717	AW869	4GB	10’ 1080P	A523	pro1
a523_pro_arm	AXP717	AW869	2GB	10’ 1080P	A523	pro1
a523_pro_arm_go	AXP717	AW869	2GB	10’ 1080P	A523	pro1
a523_aiot_demo_arm_go	AXP717	AW869	2GB	10’ 1080P	A523	aiot_demo

版权所有 © 珠海全志科技股份有限公司。保留一切权利

product_name	PMU	WIFI	RAM	LCD	CPU	Board
a523_aiot_demo_arm64	AXP717	AW869	2GB	10’ 1080P	A523	aiot_demo
a527_pro2_arm64	AXP717	AP6398S	4GB	10’ 720P	A527	pro2
a527_pro2_arm_go	AXP717	AP6398S	2GB	10’ 720P	A527	pro2
a527_aiot_demo_arm_go	AXP717	AW869	2GB	10’ 720P	A527	aiot_demo
a527_aiot_demo_arm64	AXP717	AW869	2GB	10’ 720P	A527	aiot_demo
a733_demo_aiot_arm64	AXP318	AW869C	8GB	10’ 720P	A733	demo_aiot
a733_demo_car_arm64	AXP318	AW869C	8GB	10’ 720P	A733	demo_car
t527_demo_arm_go	AXP717	AW869	2GB	10’ 720P	T527	demo
t527_demo_arm64	AXP717	AW869	2GB	10’ 720P	T527	demo
t527_demo_battery_arm_go	AXP717	AW869	2GB	10’ 720P	T527	demo
t527_demo_car_arm64	AXP717	AW869	4GB	10’ 720P	T527	demo_car
t527_pre_auto_arm64	AXP8191	NF3205Q	4GB	10’ 720P	T527V	pre_auto
t736_demo_aiot_arm64	AXP8191	AW869C	8GB	10’ 720P	T736	demo_aiot

版权所有 © 珠海全志科技股份有限公司。保留一切权利

product_name	PMU	WIFI	RAM	LCD	CPU	Board
t736_demo_car_arm64	AXP8191	AW869C	8GB	10’ 720P	T736	demo_car
a537_demo_car_arm64	AXP717B/C	AW869C	4GB	10’ 720P	A537	demo_car

3.1.4 编译单个模块

本节主要介绍各模块编译方法和更新方法。

1. 以 Settings 为例，编译：

快速编译模块：m + 任意模块名

```
示例：
$ m Settings -j40
```

2. 安装，对于 app 可以直接 install 替换：

```
adb install settings.apk（apk绝对路径）
```

3. 对于其它模块 so，bin 一类的在 $OUT 目录下有具体文件的（或者说设备内 system/vendor 目录下），我们使用 push 更新：

示例：

```
第一步：首先需要解锁设备      // 安全固件需要执行解锁操作，非安全固件无需执行解锁操作
确认Settings 开发者选项中使能 OEM unlocking为 already unlocked状态
如果没有，则需要进行如下操作
Settings 开发者选项中使能 OEM unlocking
adb reboot bootloader
fastboot oem unlock
fastboot reboot

第二步：然后执行 remount 操作：
$ adb root && adb remount       // remount设备
$ adb push test.so绝对路径  /system/lib
$ adb reboot
```

4. 编译 longan 仓库下的模块

版权所有 © 珠海全志科技股份有限公司。保留一切权利

对于只需要编译 longan，而不需要编译 Android 的情况，我司提供了一种无源码打包方式，详情见下一小节。

(1) 编译 uboot

```
./build.sh bootloader
```

3.1.5 一键编译命令

```
build --help

Usage:
example: build -sukapd...

option            option-args                          function
-h|--help         none                                 show help message
-s|--spl          none                                 build spl
-u|--uboot        none                                 build uboot
-k|--kernel       none                                 build kernel(kernel+awbsp+dts)
-l|--longan       none                                 build longan(spl+uboot+kernel)
-a|--android      none                                 build android
-r|--dist         none                                 android release build
-m|--vendor       optional, vendor-sdk-path            android grf build
-j|--job          optional, job number                 android make job number
-p|--pack         none                                 pack aw firmware
-d|--debug        none                                 pack with card0 debug port
-v|--secure       none                                 pack secure firmware
-t|--test-system  optional, dragonboard/dragonabts     pack with test system
-c|--clean        none                                 clean longan and android
```

使用前需先设置环境变量和 lunch 编译目标，例如

```
$ cd <android_top_path>/longan
$ ./build.sh config              # ----longan板型配置
$ ./build.sh                    # ----编译longan
$ cd <android_top_path>
$ source build/envsetup.sh      # ----导入环境变量
$ lunch <PRODUCT>-<VARIANT>     # ----选择对应的方案，如 lunch t736_demo_car_arm64-userdebug
$ make -j16                     # ----编译Android，j开启多核编译，服务器开发一般为服务器cpu数量的一半
$ pack <-d> <-v>                # ----打包固件
```

3.1.6 安全固件配置

Android13 行业车载方案默认是非安全固件配置，如需编译安全固件，需要修改如下：

- 需要在 Android 板型下配置文件 device-common.mk 使能安全：

```
BOARD_HAS_SECURE_OS = ture
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

- 需要在 Longan 板型下配置文件 BoardConfig.mk 打开安全编译配置：

```
LICHEE_PACK_SECURE_TYPE := any
```

3.1.7 方案编译例子

下面以 t736 编译 Android 固件的 longan 配置和 Android 配置进行说明：

3.1.7.1 T736 编译 Android 固件 longan 配置

```
android/longan$ ./build.sh config
========ACTION List: mk_config ;========
options :
All available platform:
   0. android
   1. linux
Choice [android]:
All available ic:
   0. a133
   1. a523
   2. a527
   3. a733
   4. t527
   5. t736
Choice [t736]:
All available board:
   0. demo_aiot
   1. demo_car
   2. evb1
   3. nowifi_stby_demo_aiot
   4. nowifi_stby_demo_car
   5. qa
Choice [demo_aiot]:
All available flash:
   0. default
   1. nor
Choice [default]:
```

3.1.7.2 T736 编译 Android 固件 android 配置

1. 选择方案，安卓根目录下执行:

```
   $ source build/envsetup.sh && lunch
```

1. 方案配置选择介绍说明，T736 方案配置如下:

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
Lunch menu .. Here are the common combinations:
     102. t736_demo_aiot_arm64-user
     103. t736_demo_aiot_arm64-userdebug                    //T736 demo_aiot 64非Go方案
     104. t736_demo_car_arm64-user
     105. t736_demo_car_arm64-userdebug                     //T736 demo_car 64非Go方案
     106. t736_evb1-userdebug
     107. t736_nowifi_stby_demo_aiot_arm64-user
     108. t736_nowifi_stby_demo_aiot_arm64-userdebug
     109. t736_nowifi_stby_demo_car_arm64-user
     110. t736_nowifi_stby_demo_car_arm64-userdebug
     111. t736_qa-userdebug
```

注意

T736 64 位安卓非 Go 商显方案推荐 t736_demo_aiot_arm64-userdebug 方案进行开发。
T736 64 位安卓非 Go 车载方案推荐 t736_demo_car_arm64-userdebug 方案进行开发。

3.1.7.3 生成 debug 固件

编译 android 后，pack -d 即可生成 debug 固件，该固件将串口引入卡口打印出来，配合配套的工具即可实时查看 log 信息。

3.1.8 无源码打包

在 BSP 开发过程中，为保证所提交代码能够在 Android 系统上正常运行，一般需要编译 Android 固件进行验证。Android10 以后的 SDK，源码量（包含 git 记录）保守估计在 100G 以上，且 Android 没有改动，只编译 BSP 再生成固件的过程大约耗时 2~10 分钟（视机器性能与当前负载情况）。为此，根据 Android 打包过程，提出一种无需下载 Android 代码、无需编译 Android 的打包 Android 固件的实现。

3.1.8.1 工作原理

只编译内核时，实际改动只有 bImage。Android 会利用编译好的 ramdisk.img 与 bImage 重新生成 boot.img，该过程仅依赖几个参数及工具。这些参数在同一个 Android 版本上基本是固定的从某一稳定版本完整 Android 固件中提取所有打包依赖的 Android image 镜像，直接 shell 调用相关工具，根据编译逻辑重新生成 boot.img，并替换 longan 编译所生成的 ko 文件利用生成好的 boot.img 及其他预编译好的提取得到的 image 文件，调用 longan 的打包工具，传递合适的参数，生成完整的 Android 固件。

3.1.8.2 使用

1. 先找到 pack hook，pack hook 所在目录根据由当前 sdk 的配置决定，具体值请查看/.buildconfig 文件 LICHEE_PACK_HOOK 的值。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

2. 拷贝一份稳定版本的 Android 完整固件到 pack hook 所在的目录，如/.buildconfig 中 LICHEE_PACK_HOOK=build/hook/pack/hook.sh，则固件拷贝目录为/build/hook/pack/

3.1.8.3 编译打包

```
$ ./build.sh config
$ ./build.sh
$ ./build.sh pack<_debug><_secure>
```

3.1.8.4 举例

以 Android13 为例

```
$ # cp 固件至 android13下的longan/build/hook/pack
$  cp 固件路径 longan/build/hook/pack/
$ # 选择对应配置
$ ./build.sh config
$ # 编译
$ ./build.sh
$ # 打包，debug为卡打印固件，secure 为安全固件
$ ./build.sh pack_debug_secure
```

⚠ 警告

注意：由于 *linux* 固件和 *android* 固件在内核编译时需要使用不同的编译工具链，所以在 *config* 阶段请选择 *android* 固件，并配置对应的版型。

3.1.9 PhoenixSuit 烧写固件

PhoenixSuit 是一款单路固件烧录升级工具，支持所有全志平台的固件烧录升级，支持按分区烧录等功能，更多用法和更多烧录工具参见《全志量产工具中心使用指南》。

注意

此工具为单机烧录工具，一次只能烧录一台设备，且不能与其他量产软件同时运行。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

3.2 发布

3.2.1 发布固件流程

发布固件即可用作量产使用的固件，同时也支持 OTA 升级功能。发布时需要使用该流程进行发布。

编译固件流程后，使用命令。

```
$pack4dist [-v]
```

即可生成固件及对应版本的 OTA 包。（注：上述-v 参数用于启用安全系统校验）

品牌签名：如果品牌商有自己的系统签名文件，把相关签名文件放入 vendor/security 目录。签名文件可参考 build/target/product/README 文档。

3.2.2 OTA 包

OTA 包包含差分包和完整包，以下是各个名词定义。

- 目标文件包（target-files-package）：包含固件完整的编译后目标文件的包。
- 差分包（incremental-package）：将基础版本与新版本固件之间的差别制作的补丁包。
- 完整包（full-package）：将新版本固件的完整补丁包。

OTA 包生成过程：

目标文件包（target-files-package）路径为。

```
$OUT/obj/PACKAGING/target_files_intermediates/$TARGET_PRODUCT-target_files-$DATE.zip
```

若包含签名目标文件包，则路径为。

\begin{cmdlisting} @$@OUT/signed_target_files-$DATE.zip \end{cmdlisting}

注：生成的 target_files.zip 文件需要与固件一同保存，用于后续生成 OTA 包。

完整包路径为：

```
$OUT/$TARGET_PRODUCT-full_ota-$DATE.zip
```

完整包生成命令：

```
#  ./build/tools/releasetools/ota_from_target_files [ -k vendor/security/releasekey ] target.zip
   ota.zip
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

差分包生成命令：

```
# ./build/tools/releasetools/ota_from_target_files [ -k vendor/security/releasekey ] -i origin.
  zip target.zip inc.zip
```

注：其中 vendor/security 为签名 key 放置路径，origin.zip 为基础版本（即需要升级的版本）的目标文件包，target.zip 为当前版本的目标文件包。

详细步骤参考 OTA 相关文档进行配置升级。

3.3 GKI

如需配置 GKI，则将方案目录下 (device/softwinner/jupiter) 的 CONFIG_AW_ENABLE_GKI 修改为 true。

```
diff --git a/BoardConfig.mk b/BoardConfig.mk
index 5685b75..4878eca 100644
--- a/BoardConfig.mk
+++ b/BoardConfig.mk
@@ -23,7 +23,7 @@ TARGET_BOOTLOADER_BOARD_NAME := exdroid
 TARGET_BOOTLOADER_NAME := exdroid
 TARGET_OTA_RESTORE_BOOT_STORAGE_DATA := true

-CONFIG_AW_ENABLE_GKI  ?= false
+CONFIG_AW_ENABLE_GKI  ?= true
 CONFIG_AW_GKI_VERSION := android13-5.15
```

其它 GKI 具体内容参考文档《Linux GKI 开发指南》

3.4 fastboot 烧写

获取 fastboot 工具。

1. 建议更新最新版本 Android SDK tools 中的 fastboot 工具。
2. 在 android 源代码编译过后的生成文件获得 (platform/out/host/linux-x86/bin/fastboot)。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

3.4.1 fastboot 常用命令

```
usage: fastboot [ <option> ] <command>
commands:
update <filename>                          reflash device from update.
   zip
flash <partition> [ <filename> ]           write a file to a flash
   partition
erase <partition>                          erase a flash partition
format <partition>                         format a flash partition
help                                       show this help message
```

- 常见命令烧写动态分区：

```
adb reboot fastboot
fastboot flash system system.img
fastboot reboot bootloader
fastboot flash boot boot-{Kernel-Ver}.img
fastboot reboot
```

更多场景和使用，详见《Android GMS 认证指南》

3.4.2 设备解锁

1. 确认 Settings 开发者选项中使能 OEM unlocking 为 already unlocked 状态
2. 如果没有，则从 Settings 开发者选项中先使能 OEM unlocking
3. 执行如下命令（如无 fastboot 命令，需先安装 fastboot 工具）：

```
$ adb reboot bootloader
$ fastboot oem unlock
$ fastboot reboot
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

4 方案定制

4.1 新增方案配置

4.1.1 新增 android 方案

选择参考方案，按照以下脚本输入命令。

```
# cd android
# source ./build/envsetup.sh
# lunch（选择对应平台公版方案配置）
# clone
```

根据提示输入新增方案信息。

```
please enter PRODUCT_DEVICE({device-name}): newdevice
please enter PRODUCT_NAME({product-name}): newproduct
please enter PRODUCT_BOARD({BOARD}): newboard
please enter PRODUCT_MODEL({MODEL_NAME}): NEWMODEL
please enter DENSITY(160): 320
```

完成以上操作后，将会在 device/{vendor-name}/{platform-name}/newdevice 目录新增方案配置。根据实际需求，修改方案目录中相关的 mk 文件，进行应用及参数的定制化。

4.1.2 新增 bsp 板级方案

执行完上一小节 clone 之后，会自动新增对应板级方案 longan/device/config/chips/{IC}/configs/newboard，根据实际硬件修改其中 board.dts、sys_config.fex 等信息。

注意

如硬件无修改，可不需要新增板级方案

版权所有 © 珠海全志科技股份有限公司。保留一切权利

4.2 配置分区

在 sys_partition.fex 进行分区配置，配置文件默认在 device/{vendor-name}/{platform-name}/{device-name}/system/sys_partition.fex。

super 分区为系统分区大小，包括 system、vendor、product 等镜像是包含在动态 super 分区当中，调整 super 分区即可调整系统分区大小。

```
[partition]
    name            = super
    size            = 3.5G
    downloadfile   = "super.fex"
    user_type       = 0x8000
```

data 分区大小可以由 BoardConfig.mk 文件的 BOARD_USERDATAIMAGE_PARTITION_SIZE 指定，单位是字节。

注意

一般将最后一个分区作为 *data* 分区，该分区大小是 *Nand* 或者 *eMMC* 总容量减去其他分区大小。如果需要烧写 *data* 分区镜像，分区大小需要预留一定预度，防止超出 *Nand* 或者 *eMMC* 容量。

4.3 修改启动 LOGO

启动 LOGO 为初始引导阶段的 LOGO。

将默认的启动 logo 放入位置：

```
longan/device/config/chips/{IC}/configs/{BOARD}/bootlogo.bmp
```

如有多种用途的 logo，可按照使用用途等放到该目录下，如适用于 go 版本的 logo，可将其保存在

```
longan/device/config/chips/{IC}/configs/{BOARD}/bootlogo-go.bmp
```

同时，Android 对应方案配置应添加如下配置，以表明使用该 bootlogo：

```
BOARD_ADD_PACK_CONFIG += longan/device/config/chips/$(TARGET_BOARD_IC)/boot-resource/boot-resource
    /bootlogo-go.bmp:boot-resource/bootlogo.bmp
```

4.4 修改开机动画

开机动画的目录通过如下 Makefile 变量定义：

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
device/softwinner/{PRODUCT}/common/media/config.mk:
  BOOTANIMATION_CONFIG_PATH := $(LOCAL_MODULE_PATH)/bootanimation
```

默认路径为 device/softwinner/{PRODUCT}/common/media/bootanimation/

可修改 BOOTANIMATION_CONFIG_PATH 的值切换 bootanimation 系统的找寻位置。

将动画放入：bootanimation.zip 放入以上定义的路径中。

注意

Android13 开始支持在 OTA 自动升级系统时，夜晚系统自动重启过程中降低对用户的干扰，要求这时的开机画面应该以暗色调为主。如果原始开机动画文件以亮色调为主，可另设暗色调开机动画，文件命名为 bootanimation_dark.zip，放入以上定义的路径中。

4.4.1 文件结构

bootanimation.zip 包含 part0 part1 文件夹和 desc.txt 文件，part0，part1 等文件夹里面放的是动画拆分的图片，格式为 png 或 jpg。

```
|- desc.txt
|- audio_conf.txt
|- part0
    |- part000.png
    |- part001.png
    |- part0….png
    |- audio.wav(可选)
    |- trim.txt(可选)
|- part1
    |- part100.png
    |- part101.png
    |- part1….png
    |- audio.wav(可选)
    |- trim.txt(可选)
|- part...N
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

4.4.2 desc.txt 配置文件

```
第一行：
WIDTH HEIGHT FPS
后面每行，表示部分part动画：
TYPE: 类型（p：播放直到开机完成，c：播放完整动画）
COUNT: 循环次数，0表示无限循环直到开机结束
PAUSE: part结束后暂停帧数
PATH: 文件加路径（如：part0）
RGBHEX: （可选）背景颜色：#RRGGBB
CLOCK: （可选）画当前时间的y坐标(for watches)
```

例如：

```
800 480 15
p 1 0 part0
p 0 0 part1
```

说明：第一行：800 为宽度，480 为高度，15 为帧数。第二行开始 p 为标志符，接下来第二列为循环次数（0 为无限循环），第三项为两次循环之间间隔的帧数，第四项为对应的目录名。播放动画时会按照图片文件名顺序自动播放。

4.5 修改开机音乐

如需开机音乐，将开机音乐放入 part0 目录中，命名为 audio.wav。在根目录中加入 audio_conf.txt，复制原有动画配置即可。

4.5.1 优化 png 图片

由于图片占用内存较大，需要做一些优化来减少图片资源占用及加快读取时间，可参考源码中 frameworks/base/cmds/bootanimation/FORMAT.md 文件进行优化，如下命令可以无损压缩图片。

```
for fn in *.png ; do zopflipng -m ${fn} ${fn}.new && mv -f ${fn}.new ${fn}; done
```

4.5.2 打包 bootanimation.zip

windows 使用 winrar 打包，选择 ZIP 格式，压缩标准要选 “储存”；

版权所有 © 珠海全志科技股份有限公司。保留一切权利

linux 系统下使用命令：

```
$ zip -0qry -i \*.txt \*.png \*.wav  bootanimation.zip *.txt part*。
```

⚠ 警告

linux 命令使用*-0* 指定压缩等级为最低等级 *stored*，即只归档不压缩，否则可能由于包格式问题引起动画显示为黑屏。

4.6 修改充电图标

在 android 目录下执行。

```
python bootable/recovery/interlace-frames.py battery1.png battery2.png ... batteryn.png
    battery_scale.png
```

然后将生成的 battery_scale.png 替换 device/softwinner/common/health/images/目录下 battery_scale.png 。其中 [battery1.png battery2.png … batteryn.png] 为充电动画的图标。
如图片数量有变化则需修改配置 device/softwinner/common/health/animation.txt。

4.7 定制 recovery 功能

Recovery 是 Android 的专用升级模式，用于对 android 自身进行更新；进入 recovery 模式的方法是，在 android 系统开机时，按住一个特定按键，则会自动进入 android 的 recovery 模式。

4.7.1 按键选择

通常情况下，一块方案板上的按键个数不同，或者排列不同，这都导致了方案商在选择作为开机阶段 recovery 功能的按键有所不同。因此，系统中提供了一种方法用于选择进入 recovery 模式的按键。

在 longan/device/config/chips/{IC}/configs/{BOARD}/uboot-board.dts 配置脚本中，提供了一项配置，用于选择按键的键值，如下所示。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
&fastboot_key {
    device_type = "fastboot_key";
    key_max = <42>;
    key_min = <38>;
};

&recovery_key {
    device_type = "recovery_key";
    key_max = <31>;
    key_min = <28>;
};
```

它表示，所选择用于作为 recovery 功能的按键的键值范围落在 key_min 到 key_max 之间，即 4 到 6 之间。由于所有按键的选择都可以通过前面介绍的方法查看，因此，假设用户要选的按键是 a，用户这里选择配置的方法是。

1. 按照前面介绍的方法，读出所有按键的键值；
2. 读出 a 的键值 a1，同时取出两个相邻于 a 的键值，记为 b1 和 c1，b1>c1；
3. 计算出 (a1 + b1)/2，(a1 + c1)/2，分别填写到 key_max 和 key_min 处；
4. 如果 a1 刚好是所有按键的最小值，则取 key_min 为 0；如果 a1 刚好是所有按键的最大值，则取 key_max 为 63；

经过以上的步骤，就可以选择一个特定的按键进入 recovery 模式。取了一个平均值的原因是考虑到长时间的使用，电阻的阻值可能会略有变化导致键值变化，取范围值就可以兼容这种阻值变化带来的键值变化。

在系统启动时，按住设定的特定按键进入 recovery 模式，进入该模式后，可以选择升级文件升级。

4.8 方案文件替换

Android 方案中有时候需要定制化一些需求，如替换开机 LOGO、env.cfg、分区配置等，除了可在 longan 目录下对应的路径下替换外，android 方案也提供了接口进行替换。其原理是在 pack 操作时，将需要替换的文件拷贝到 longan/out/pack_out 目录下。

在 android 方案中通过 BOARD_ADD_PACK_CONFIG 配置来指定需要替换的文件。

```
# BoardConfig.mk
BOARD_ADD_PACK_CONFIG += $(TARGET_DEVICE_DIR)/system/sys_partition.fex
BOARD_ADD_PACK_CONFIG += $(PRODUCT_PLATFORM_PATH)/common/system/b1_env.cfg:env.cfg
```

通过这种方式替换的文件只能是会被拷贝到 longan/out/pack_out 目录下的文件。格式为：`local_filepath[:copy_filename]`。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

4.9 overlay 说明

Android overlay 机制允许在不修改 apk 或者 framework 源代码的情况下，实现资源的定制。
以下几类能够通过 overlay 机制定义。

1. Configurations (string, bool, bool-array)
2. Localization (string, string-array)
3. UI Appearance (color, drawable, layout, style, theme, animation)
4. Raw resources (audio, video, xml)
   更详细的资源文件可浏览android 网站

4.9.1 为产品添加 Overlay 目录

有两种不同的 overlay 目录定义。

1. PRODUCT_PACKAGE_OVERLAYS
   用于指定产品。
2. DEVICE_PACKAGE_OVERLAYS
   用于同一设备模型的一系列产品。

如果包含同一资源，那么 PRODUCT_PACKAGE_OVERLAYS 的内容将覆盖 DEVICE_PACKAGE_OVERLAYS 中的内容。
如果要定义多个 overlays 目录，需要用空格隔开，同一资源的定义，将使用先定义的目录中的资源。

在方案目录下创建 common/overlay 和 {device-name}/overlay 目录，分别用于 device 通用及 product 使用的 overlay 文件夹。

4.9.2 改变 mk 文件来添加 overlays 的编译项

在文件 common/overlay/config.mk 和 {device-name}/overlay/config.mk 中分别添加。

```
DEVICE_PACKAGE_OVERLAYS := \
    $(LOCAL_MODULE_PATH)/overlay \
    $(DEVICE_PACKAGE_OVERLAYS)

PRODUCT_PACKAGE_OVERLAYS := \
    $(LOCAL_MODULE_PATH)/overlay \
    $(PRODUCT_PACKAGE_OVERLAYS)
```

注：必须加上 $(PRODUCT_PACKAGE_OVERLAYS) 变量否则将找不到默认资源。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

4.10 预装 APK

预装 apk 安装有两种方法，可以安装到 system/app 目录下，也可以安装到 system/preinstall 目录下。

注：apk 名字不能含有中文、空格等特殊字符。

由于涉及版权问题，建议不安装 GAPP 应用。若通过 GMS 认证需安装 Google 提供的正版 GAPP 应用。

4.10.1 预装到 system/app 目录

1. 在目录 vendor/aw/public/prebuild/apk/中创建一个目录存放对应 APK。
2. 将 apk 放入该目录中。
3. 在该目录中创建 Android.mk 文件，并编辑。

```
# Example
LOCAL_PATH := $(call my-dir)
include $(CLEAR_VARS)
LOCAL_MODULE := APK_MODULE_NAME（模块的唯一名字）
LOCAL_MODULE_CLASS := APPS
LOCAL_MODULE_TAGS := optional
LOCAL_BUILT_MODULE_STEM := package.apk
LOCAL_MODULE_SUFFIX := $(COMMON_ANDROID_PACKAGE_SUFFIX)
LOCAL_CERTIFICATE := PRESIGNED（签名方式）
#LOCAL_OVERRIDES_PACKAGES := OVERRIDES_MODULE（要替代的模块）
LOCAL_SRC_FILES := name.apk（apk的文件名，一般与MODULE同名）
include $(BUILD_PREBUILT)
```

4. 在方案 mk 文件（device/{vendor-name}/{platform-name}/{product-name}.mk）中 PRODUCT_PACKAGES 项中加入。

```
PRODUCT_PACKAGES += APK_MODULE_NAME（apk模块名字，预装多个apk用空格隔开）
```

4.10.2 预装到 system/preinstall 目录

1. 同预装到 system/app 目录，完成所有步骤。
2. 修改 apk 目录下的 Android.mk，加入一行。

```
LOCAL_MODULE_PATH := $(TARGET_OUT)/preinstall
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

5 模块配置

5.1 自定义按键配置

5.1.1 Android 按键功能的映射

详细配置参考《Linux_LRADC_开发指南》，需要修改方案对应的 board.dts 中，主要修改 key_cnt 及对应电压值。

```
keyboard {
    compatible = "allwinner,keyboard_1350mv";
    status = "okay";
    key_cnt = <3>;
    key0 = <475 0x7372>;
    key1 = <646 0x73>;
    key2 = <900 0x72>;
};
```

5.2 LCD Panel 配置

详细配置请参考文档 《Linux LCD 开发指南》。

5.3 Touch Panel 配置

发布的 SDK 中，默认有对 GT81x/GT82x/GT9xx/GT9xxf（汇顶）、GSLx680(思立微) 等 ctp 的支持。

5.3.1 配置文件的修改

配置文件目录：longan/device/config/chips/{IC}/configs/{BOARD}/board.dts。

详细配置请参考《Linux Device Tree 使用指南》。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

5.3.2 Android 层的配置修改

在 device/{vendor-name}/{platform-name}/{device-name}/input/init.input.rc 文件中加入装载驱动模块的语句。

```
insmod /vendor/lib/modules/gslX680new.ko
```

5.4 G-Sensor 配置

配置文件目录：longan/device/config/chips/{IC}/configs/{BOARD}/board.dts。

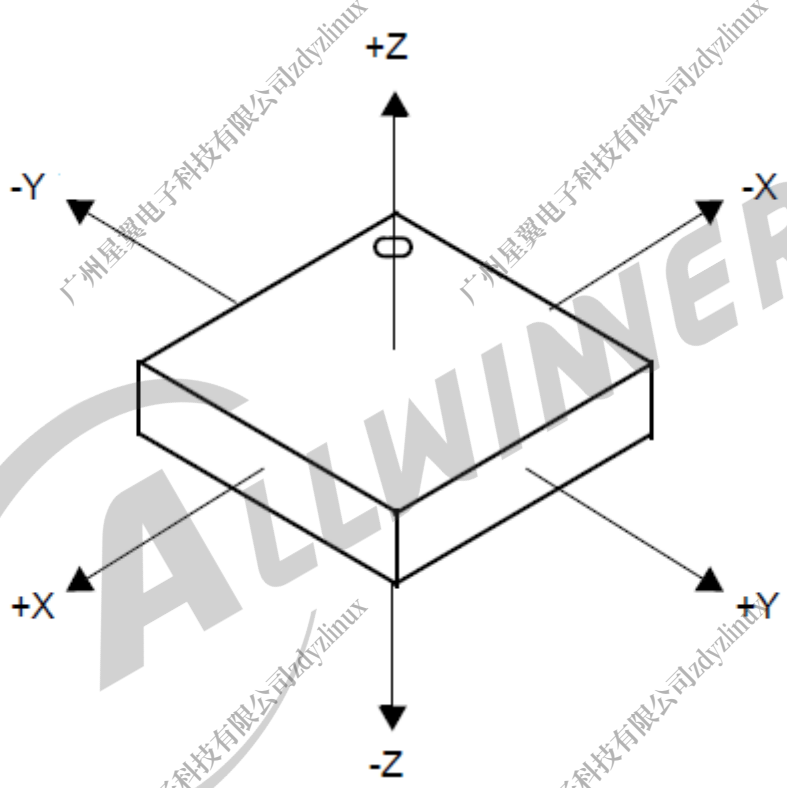

图 5-1: gsensor

5.4.1 Android 层配置修改

以 bma250 为例。

Sensor 驱动是自动加载的，自动加载部分配置好后，不需要手动 insmod。

1) 方向的调整。

在 device/{vendor-name}/{platform-name}/{device-name}/input/gsensor.cfg 中，以 bma250 方向为例进行说明。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
;name: bma250
gsensor_name = bma250   //标示用bma250c gsensor
gsensor_direct_x = false       //如果x轴反向，则置false
gsensor_direct_y = false       //如果y轴反向，则置false
gsensor_direct_z = false       //如果z轴反向，则置false
gsensor_xy_revert = true      //如果x轴当y轴用，y轴当x轴，则置true
```

Gsensor 方向调试说明。假定机器的长轴为 X 轴，短轴为 Y 轴，垂直方向为 Z 轴。

5.4.1.1 首先调试 Z 轴

1. 第一步观察现象。

旋转机器，发现当只有垂直 90° 时或者是在旋转后需要抖动一下，方向才会发生变化，则说明 Z 轴反了。若当机器大概 45° 拿着的时候也可以旋转，说明 Z 轴方向正确。无需修改 Z 轴方向。

2. 第二步修改 Z 轴为正确方向。

此时需要找到当前使用模组的方向向量（根据模组的名称）。如果此时该方向 Z 轴向量（gsnesor_direct_z）的值为 false，则需要修改为 true；当为 true，则需要修改为 false。通过 adb shell 将修改后的 gsnesor.cfg 文件 push 到 system/usr 下，重启机器，按第一步观察现象。

5.4.1.2 其次查看 X，Y 轴是否互换

1. 第一步观察现象。

首先假定长轴为 X 轴，短轴为 Y 轴，以 X 轴为底边将机器立起来。查看机器的 X，Y 方向是否正好互换，若此时机器的 X，Y 方向正好互换，在说明需要将 X，Y 方向交换。若此时 X，Y 方向没有反置，则进入 X，Y 方向的调试。

2. 第二步交换 X，Y 方向

当需要 X，Y 方向交换时，此时需要找到当前使用模组的方向向量（根据模组的名称）。如果此时该 X，Y 轴互换向量（gsensor_xy_revert）的值为 false，则需要修改为 true，当为 true，则需要修改为 false。通过 adb shell 将修改后的 gsnesor.cfg 文件 push 到 system/usr 下，重启机器，按第一步观察现象。

5.4.1.3 再次调试 X，Y 轴方向

1. 第一步观察现象。首先假定长轴为 X 轴，短轴为 Y 轴，以 X 轴为底边将机器立起来，查看机器的方向是否正确，如果正确，说明长轴配置正确，如果方向正好相反，说明长轴配置错误。将机器旋转到短轴，查看机器方向是否正确，如果正确，说明短轴配置正确，如果方向正好相反，说明短轴配置错误。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

2. 第二步修改 X，Y 轴方向。

当需要修改 X，Y 轴方向时，当只有长轴方向相反或者是只有短轴方向相反时，则只修改方向不正确的一个轴，当两个方向都相反时，则同时修改 X 与 Y 轴方向向量。找到当前使用模组的方向向量（根据模组的名称）。
若长轴方向相反，如果此时该方向 X 轴向量（gsnesor_direct_x）的值为 false，则需要修改为 true，当为 true，则需要修改为 false。

若短轴方向相反，如果此时该方向 Y 轴向量（gsnesor_direct_y）的值为 false，则需要修改为 true，当为 true，则需要修改为 false。

通过 adb shell 将修改后的 gsnesor.cfg 文件 push 到 system/usr 下，重启机器，按第一步观察现象。若发现还是反向 X 轴或者 Y 轴的方向仍然相反，则说明 X 轴为短轴，Y 轴为长轴。此时：

若长轴方向相反，如果此时该方向 Y 轴向量（gsnesor_direct_y）的值为 false，则需要修改为 true，当为 true，则需要修改为 false。

若短轴方向相反，如果此时该方向 X 轴向量（gsnesor_direct_x）的值为 false，则需要修改为 true，当为 true，则需要修改为 false。

5.5 显示配置

详细配置请参考《Android Display 使用指南》。

5.6 WiFi/BT 配置

详细配置请参考《Android 13 Wi-Fi/BT 开发指南》。

5.7 Camera 配置

详细配置请参考《Android Camera 配置指南》。

5.7.1 Camera 驱动加载顺序

在文件 device/{vendor-name}/{platform-name}/{device-name}/camera/init.camera.rc 添加 camera 驱动 ko 文件加载顺序如下。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
on boot

### csi module

    insmod /vendor/lib/modules/videobuf2-memops.ko
    insmod /vendor/lib/modules/videobuf2-dma-contig.ko
    insmod /vendor/lib/modules/vin_io.ko
    insmod /vendor/lib/modules/gc5035_mipi.ko
    insmod /vendor/lib/modules/gc02m2_mipi.ko
    insmod /vendor/lib/modules/ov2680_mipi.ko
    insmod /vendor/lib/modules/actuator.ko
    insmod /vendor/lib/modules/dw9714_act.ko
    insmod /vendor/lib/modules/vin_v4l2.ko
```

5.7.2 Camera 参数配置

配置文件路径：device/{vendor-name}/{platform-name}/{device-name}/camera/camera.cfg。

media_profiles.xml 的路径：

```
device/{vendor-name}/{platform-name}/{device-name}/camera/
   media_profiles.xml
```

内容简介：该文件主要保存 Camera 支持的摄像相关参数，包括摄像质量，音视频编码格式、帧率、比特率等等，该参数主要有摄像头厂商提供。需要注意帧率配置，我们配置 frameRate="24" 为 24 帧，这个是多媒体要求的 camera 帧率最低的要求，这样配置我们可以满足低性能的 sensor，适用的 sensor 范围广一些。

5.7.3 配置文件的修改

配置文件路径：longan/device/config/chips/{IC}/configs/{BOARD}/board.dts。

根据原理图进行相关配置参数的修改。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

6 Android 系统配置

6.1 壁纸设置

6.1.1 默认壁纸

```
替换文件frameworks/base/core/res/res/drawable-swxxxdp-nodpi/
   default_wallpaper.png
```

注意

可通过 *overlay* 方式替换。

6.1.2 添加壁纸

准备壁纸及壁纸的缩略图放进壁纸存放目录 packages/apps/WallpaperPicker2/res/drawable-nodpi，并按照文件夹内文件命名，分别为 wallpaper_xxx.png 与 wallpaper_xxx_small.png。

在 platform/packages/apps/WallpaperPicker2/res/values-nodpi/wallpapers.xml 中添加该壁纸索引，如下。

```
<resources>
    <string-array name="wallpapers" translatable="false">
        <item>wallpaper_00</item>
        <item>wallpaper_01</item>
        ……
        <item>wallpaper_xxx</item>
    </string-array>
</resources>
```

注：可通过 overlay 方式修改，具体请参照 2.1overlay 说明。

6.2 Launcher 默认图标和快捷栏设置

修改文件 platform/packages/apps/Launcher3/res/xml/default_workspace_5x6.xml，文件中各配置项含义如下。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

设置名	意义
packageName	所运行的 APP 的 package 名。
className	该 APP 的 Activity 的 class 名 (packageName.ActivityName)。
screen	表示在第几个，根据显示的个数决定。
x	放在该屏的第几行。
y	放在该屏的第几列。

表 6-1: Launcher 快捷栏配置表

注：可通过 overlay 方式修改，具体请参照 2.1overlay 说明。

6.3 后台服务管理配置

6.3.1 功能介绍

awbms，全志自研的后台管理服务，使得多应用运行场景下还保留足够的空闲内存，当系统资源内存紧张时，通过清除后台服务，让系统保持运行流畅。

6.3.2 方案配置

注意

通过 *device/softwinner/common/config/awbms_config* 复制到 *$(TARGET_COPY_OUT_VENDOR)/etc/*开启后台管理服务。通过配置文件中各项修改后台服务管理的配置。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
debug: false
limit: 12
threadCheck: true
memoryCheck: true
memoryLimit: 450,0
memoryTrim: false
stopTaskTime: 60
skipService: true
blockBroadcast: false
lmk_level: 100,150,250

whitelist:
android
com.android
com.android.phone
```

- debug 是否打开调试打印，取值 true/false，默认为 false。
- limit 限制后台个数，清理多余的后台，-1 则不限制，默认为 0。
- threadCheck 检查优化关键进程优先级，取值 true/false，默认为 false。
- memoryCheck 根据当前内存情况清理后台应用，取值 true/false，默认为 false。
- stopTaskTime: 熄屏多长时间后，对运行任务做清理，单位：秒；负值禁用
- skipService 是否跳过非白名单服务自动启动，取值 true/false，默认为 true。
- whitelist 配置包名前缀白名单，系统白名单的进程不会被上述机制清理。
- 当前输入法、正在播放音乐进程等自动系统白名单，无须单独配置。

6.3.3 用户设置

通过：设置-> 系统-> 后台管理，可以进入设置后台清理列表。列表中只显示不在系统配置白名单中的应用。勾选后的应用添加到用户白名单中。

6.3.4 功能调试

通过删除 awbms_config 配置文件可以关闭后台管理功能，通过以下命令可开启/关闭相关调试打印：

```
service call background 1 i32 1/0
```

logcat 中关于 BackgroundManagerService 相关的打印。

```
D bms: skipped service Intent(xxx)
I bms: forceStopPackage com.xxx.xxx
D bms: killBackgroundProcesses com.xxx.xxx
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

当看到则 forceStopPackage 或 killBackgroundProcesses 表示后台应用被后台服务管理清除，如有必要则需要将应用加入白名单或者在设置中取消勾选清除列表。

使用命令：dumpsys background 可以查看后台服务当前的状态。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

7 AIOT 方案配置

AIOT android13 SDK 主要面向行业车载方向，与平板方案相比，会在 camera 和显示模块实现较多的差异化需求和配置。

7.1 多屏方案配置

详细配置请参考文档《AIOT Android13 多屏方案开发指南》。

7.2 双触摸配置

详细配置请参考文档《Android13 双触摸方案开发指南》。

7.3 Camera 配置

详细配置请参考文档《T527 Android13 TP2815 使用指南》，《T527 Android13 NVP6158 使用指南》，《AIOT Android13 awCamera API 使用指南》和《AIOT Android13 DP_IN 方案开发指南》。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

8 AAOS 系统配置

Android Automotive os(简称 AAOS) 是一个完整、独立的、基于 Android 平台的车载信息娱乐系统。

8.1 EVS 配置

详细配置请参考《T527 Android13 EVS 开发指南》。

8.2 音频配置

AAOS 音频配置与 Android 系统有一定的差异，需要支持独立控制喇叭输出不同声音。

详细配置请参考《T527V Android13 音频开发指南》。

广州星翼电子科技有限公司zdyzlinux

9 工具使用

9.1 DragonRunin

DragonRunin 是自动化老化测试的 app，可通过 LED 板卡显示测试结果，当前支持待机唤醒、循环重启、DDR 测试、DDR 性能测试、视频老化、GPU 老化。

测试项	说明
待机唤醒	设备休眠唤醒测试，可设置休眠次数、休眠时间、唤醒时间等。
循环重启	设备循环重启测试，可设置重启时间、重启次数、是否恢复出厂等。
DDR 测试	设备 DDR 老化测试，可校验设备 DDR 是否正常。
DDR 性能测试	设备 DDR 性能测试。
视频老化	测试设备视频播放能力。
GPU 老化	测试设备 GPU 运算能力。

表 9-1: Keeptesting 测试项

在 Android13 中已经在 userdebug 固件默认集成了 DragonRunin 应用，里面已经自带了基础的测试用例。工具默认路径为：

```
vendor/aw/public/package/apk/DragonRunin
```

9.1.1 自动化

通过属性 **ro.runin.auto_run_mode** 可以配置开机自动运行测试，在方案配置如 **device/softwinner/common/common.mk** 中添加以下配置：

```
PRODUCT_SYSTEM_DEFAULT_PROPERTIES += \
ro.runin.auto_run_mode=every
# 参数有三类：
# every：每次开机后自动运行测试
# 1：仅第一次开机后自动运行测试
# 空：不自动运行测试
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

9.1.2 手动执行

手动执行可分两种：

1. UI 启动

设置 -> 关于平板电脑 -> 快速点击多次 “版本号” -> 计算器输入 “777++”

注：如果有弹窗请求权限，请选择允许。

2. adb shell 指令启动

```
pm grant com.softwinner.runin android.permission.READ_EXTERNAL_STORAGE
pm grant com.softwinner.runin android.permission.
   WRITE_EXTERNAL_STORAGE
pm grant com.softwinner.runin android.permission.CAMERA
pm grant com.softwinner.runin android.permission.RECORD_AUDIO
pm grant com.softwinner.runin android.permission.ACCESS_FINE_LOCATION
pm grant com.softwinner.runin android.permission.
   ACCESS_COARSE_LOCATION
pm grant com.softwinner.runin android.permission.ACCESS_MEDIA_LOCATION
pm grant com.softwinner.runin android.permission.BLUETOOTH_CONNECT
pm grant com.softwinner.runin android.permission.BLUETOOTH_SCAN
am start -n com.softwinner.runin/com.softwinner.runin.activities.
   RuninActivity
```

9.1.3 测试结果

全部测试结束且都通过测试时，会在 UI 页面显示 “PASS” 弹窗。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

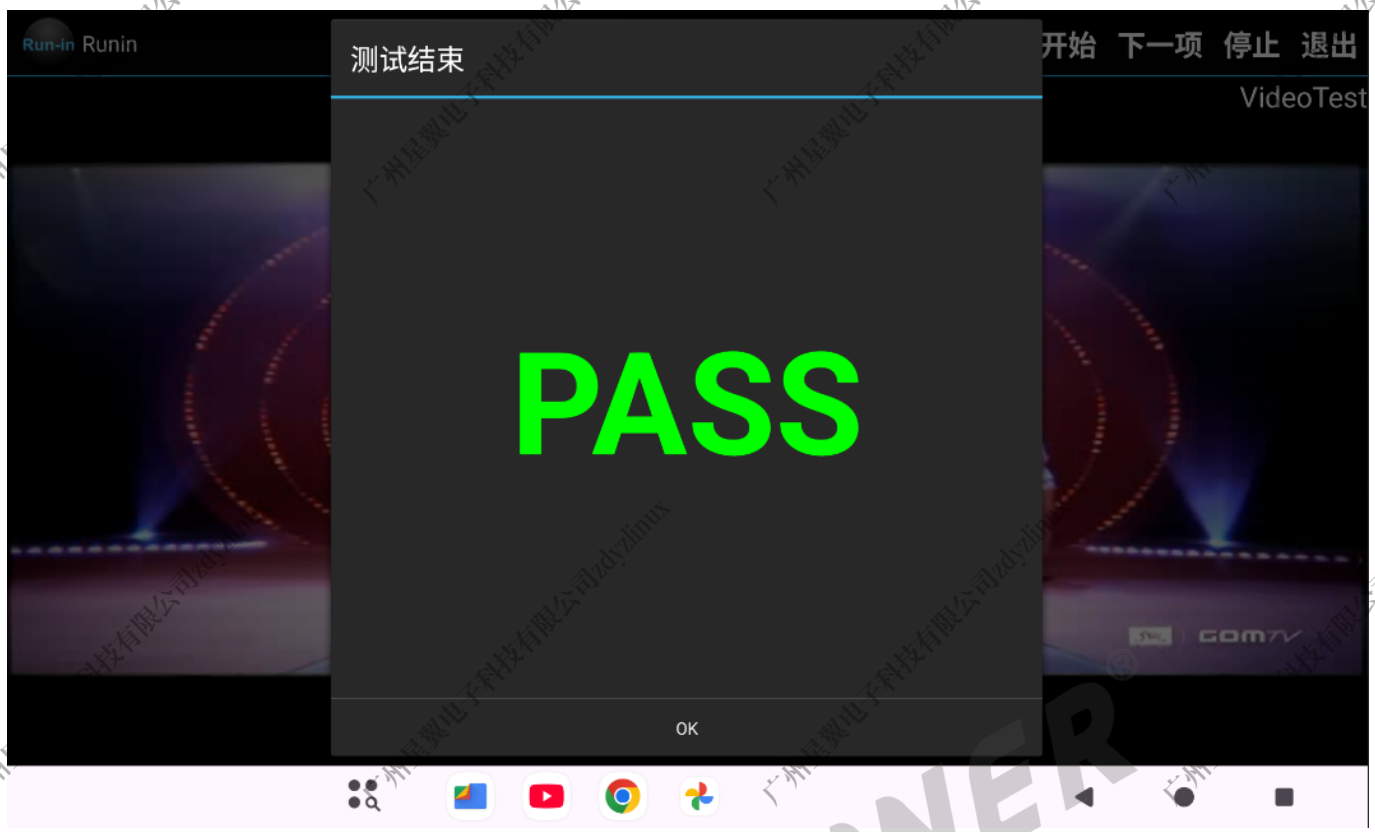

图 9-1: 测试 Pass

全部测试结束且存在失败项时，会在 UI 页面显示 “FAIL” 弹窗。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

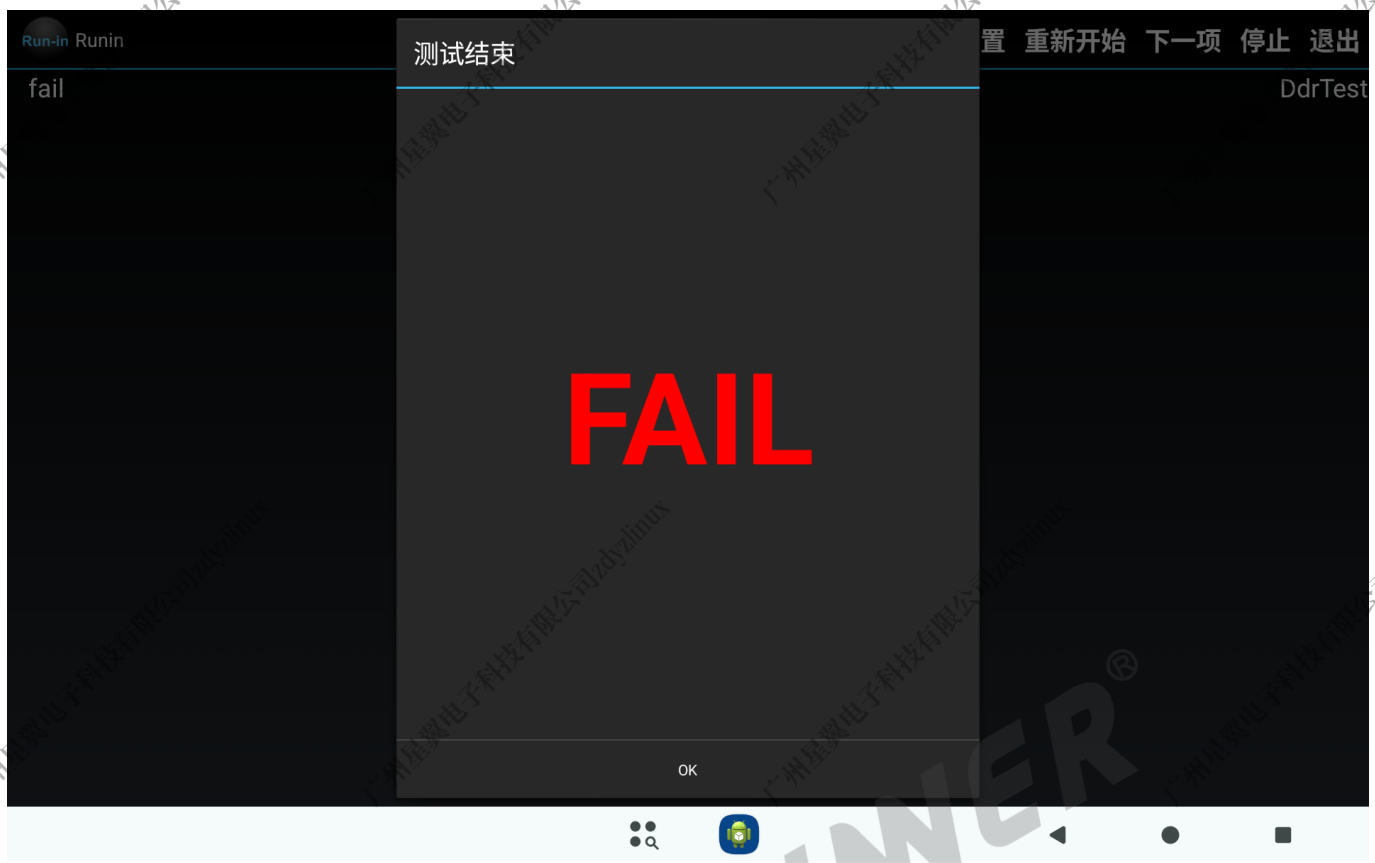

图 9-2: 测试 Fail

注意

- *LED* 功能需要合入相关补丁，详情可咨询 *AE*。

9.2 Keeptesting

Keeptesting 是用于老化测试的 app，当前支持待机唤醒、循环重启、循环 OTA 升级、读写测试、旋屏测试、DDR 测试、WIFI 测试、蓝牙测试、Monkey 测试、DDR 性能测试。

打开方式如下：

```
AOSP计算器中输入
999++
也可以使用adb shell命令行：
am start -n com.clock.pt1.keeptesting/.MainActivity
```

注意

- *GMS* 方案中主界面显示的是 *Google* 计算器，通过以上操作无法打开动态开关。
- *AOSP* 计算器默认隐藏，打开方式为 *Settings->About tablet*, 点击 *5* 次 *Version number*。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

测试项	说明
待机唤醒	设备休眠唤醒测试，可设置休眠次数、休眠时间、唤醒时间等。
循环重启	设备循环重启测试，可设置重启时间、重启次数、是否恢复出厂等。
循环 OTA 升级	设备循环 OTA 测试，需要将升级包推至/data/ota_package 目录，需要 777 授权并重命名为 update.zip。
读写测试	设备读写性能和读写老化测试，可单独测试设备 IO 性能。
旋屏测试	设备屏幕旋转测试，可设置旋转度数、旋转方向、时间间隔等。
DDR 测试	设备 DDR 老化测试，可校验设备 DDR 是否正常。
WIFI 测试	设备 WiFi 老化测试。
蓝牙测试	设备蓝牙老化测试。
Monkey 测试	设备 Monkey 老化测试。
DDR 性能测试	设备 DDR 性能测试。

表 9-2: Keeptesting 测试项

9.3 DragonAtt

DragonAtt 是用于板卡测试和老化测试的 app，当前支持 28+ 板卡测试用例，12+ 老化测试用例，详情见《DragonAtt 测试工具使用指南》。

部分板卡用例如下：

测试项	说明
版本测试	读取系统各类版本信息。
内存测试	获取当前系统内存和存储大小。
WiFi 测试	扫描附近 WiFi，支持过滤特定 WiFi 信号强度，测试设备 WiFi 功能。
蓝牙测试	扫描附近蓝牙设备，测试设备蓝牙功能。
Gsensor 测试	测试设备重力传感器功能。
录音测试	测试设备录音功能，当前测试方法为先录后播。
扬声器测试	测试设备扬声器功能。
Video 测试	测试设备视频播放能力。
触屏测试	测试当前设备屏幕是否有坏点。
Camera 测试	测试当前设备相机功能。

表 9-3: DragonAtt 测试项

版权所有 © 珠海全志科技股份有限公司。保留一切权利

10 系统调试

10.1 Awlog 日志

AwLog 是全志研发日志系统版，默认抓取系统日志，存放于/data/media/awlog 目录下，可从设备主界面 Awlog 应用进入，支持实时保存内核 Log 及 Android logcat 到存储中，详情见《Android 13 日志使用指南》。

10.2 adb 工具

Android 调试桥 (adb) 是一种功能多样的命令行工具，可让您与设备进行通信。adb 命令可用于执行各种设备操作，例如安装和调试应用。adb 提供对 Unix shell（可用来在设备上运行各种命令）的访问权限。它是一种客户端-服务器程序，包括以下三个组件：

10.2.1 常用命令

选项	说明
adb devices -l	查看连接到计算机的 Android 设备或者模拟器。
adb install <apk 文件路径 >	安装 apk 到设备上，对于出现 App package must target，需要添加–bypass-low-target-sdk-block 参数。
adb uninstall <packagename>	快速卸载安装的第三方应用。
adb pull < 远程路径 > < 本地路径 >	从设备下载文件到电脑。

表 10-1: adb 常用命令

10.3 frame_monitor 工具

frame_monitor 是全志独立研发的查看不同图层帧率的工具，该工具弥补了对帧率体验评测的空缺。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

10.3.1 常用命令

```
frame_monitor可缺省[-h][-ls][-a][-l layer_name][-t period_s][-
    frecord_file]
```

选项	说明
-ls，–layer_list	显示当前活动的 layer 信息，方便针对性监控需要的图层。
-l，–layer	监控具体的 layer，需指定完成 layer 名称，可缺省；缺省时会提供图层选择。
-t，–interval	采样周期 Default:0.5s，与显示的 Vsync 周期无关。
-f，–file	监控信息输出到文件。
-a，–all	全面监控，会将在一次采样周期内监测所有图层的帧信息。。
-h，–help	帮助提示信息。

表 10-2: frame_monitor 常用命令

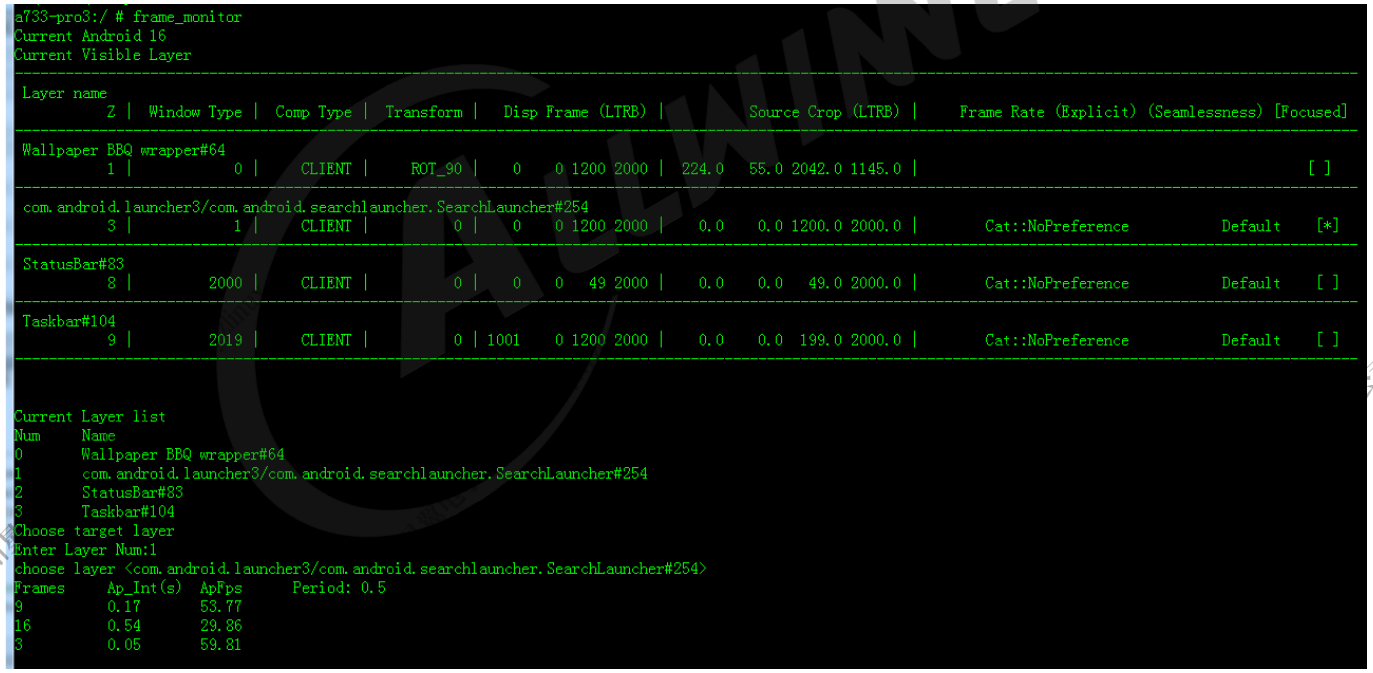

图 10-1: frame_monitor

10.4 cpu_monitor 工具

frame_monitor 是全志独立研发的查看系统负载的工具，常用于性能相关场景分析。可以快速一览当前 cpu 负荷。

```
127|a537-pro:/ # cpu_monitor
-----------------------------------------------------a537 CPU[x]<Freq:MHZ>-<Usage:%>-----------------------------------------------------
     CPU0        CPU1        CPU2        CPU3        CPU4        CPU5        CPU6        CPU7   TempL   TempB   TempM  GPU  GPUTemp  NPUTemp   DDR  Times
  | 912   0%| | 912   3%| | 912   0%| | 912   0%| | 408   3%| | 408   0%| | 408   0%| | 408   0%| 43092   42903   43596  200   43344    -255    150      0
  | 792   0%| | 792   2%| | 792   0%| | 792   0%| | 408   0%| | 408   0%| | 408   0%| | 408   0%| 43533   42714   43281  200   43785    -255    150      1
  | 792   0%| | 792   1%| | 792   0%| | 792   0%| | 408   0%| | 408   0%| | 408   0%| | 408   0%| 43659   42588   43218  200   43470    -255    150      2
  | 408   0%| | 408   2%| | 408   0%| | 408   0%| | 408   0%| | 408   0%| | 408   0%| | 408   0%| 43407   42840   43029  200   43722    -255    150      3
  | 408   1%| | 408   2%| | 408   0%| | 408   0%| | 408   0%| | 408   0%| | 408   0%| | 408   0%| 43785   42777   43155  200   43470    -255    150      4
  | 408   0%| | 408   3%| | 408   1%| | 408   0%| | 408   0%| | 408   0%| | 408   0%| | 408   0%| 43281   42903   43092  200   43722    -255    150      5
  | 408   1%| | 408   2%| | 408   1%| | 408   0%| | 408   0%| | 408   0%| | 408   0%| | 408   0%| 43596   42966   43218  200   43974    -255    150      6
```

图 10-2: cpu_monitor

10.5 mtop 工具

mtop 是全志独立研发的查看系统带宽的工具，也常用于性能相关场景分析。

```
a733-pro3:/sdcard # mtop -d 2 -m
set devfreq governor node 0444!
set min_freq to max_freq value!

iter: -1
delay: 2.000000
unit: MB
output:


Max:57
total     caltot    cpu0      cpu1      gpu       npu       de        eink      ve0       ve1       ve2       g2d       isp       csi
57.00     56.00     29.00     27.00     0.00      0.00      0.00      0.00      0.00      0.00      0.00      0.00      0.00      0.00
53.00     53.00     27.00     26.00     0.00      0.00      0.00      0.00      0.00      0.00      0.00      0.00      0.00      0.00
175.00    175.00    89.00     86.00     0.00      0.00      0.00      0.00      0.00      0.00      0.00      0.00      0.00      0.00
59.00     58.00     30.00     28.00     0.00      0.00      0.00      0.00      0.00      0.00      0.00      0.00      0.00      0.00
68.00     68.00     35.00     33.00     0.00      0.00      0.00      0.00      0.00      0.00      0.00      0.00      0.00      0.00
```

图 10-3: mtop

10.5.1 常用命令

```
mtop [-d delay] [-m] [-o FILE] [-h]
```

选项	说明
mtop -d [time]	间隔多久打印一次，单位秒。
mtop -m	转化为 MB 显示。
mtop -o [filepath]	输出到文件中，mtop -o /sdcard/mtop.log。
mtop -h	帮助提示信息。

表 10-3: mtop 常用命令

版权所有 © 珠海全志科技股份有限公司。保留一切权利

10.6 dumpsys 功能

dumpsys 是一种在 Android 设备上运行的工具，可提供有关系统服务的信息。可以使用 Android adb 从命令行调用 dumpsys，获取在连接的设备上运行的所有系统服务的诊断输出

10.6.1 常用命令

```
adb shell dumpsys [-t timeout] [--help | -l | --skip services | service [arguments] | -c | -h]
```

选项	说明
dumpsys -help	输出 dumpsys 工具的帮助信息。
dumpsys -l	输出支持 dumpsys 的系统服务的完整列表。
dumpsys [service]	输出指定服务的信息。例如，dumpsys SurfaceFlinger。

表 10-4: dumpsys 基础命令

10.6.2 进阶

部分服务支持 dumpsys 添加额外参数，比如-h，-c，-t，–skip 等等，可以获取更加丰富的信息。

选项	说明
dumpsys -t timeout [service]	输出 dumpsys 工具的帮助信息。例如，dumpsys -t 100000 meminfo
dumpsys –skip [services]	指定不希望包含在输出中的 services。dumpsys 直接运行会输出所有服务，skip 可以帮助过滤部分。
dumpsys service] -c	对于某些服务，支持以计算机可读的格式输出数据。
dumpsys service] -h	对于某些服务，支持查看服务的帮助信息以及其他选项。

表 10-5: dumpsys 进阶命令

10.7 crashdump 模式

CrashDump 是全志平台的一个分析内核崩溃的工具。当 Linux 系统**内核发生崩溃的时候**，可以通 KEXEC+KDUMP 等方式收集内核崩溃之前的内存，生成一个转储文件 vmcore。内核开发者通过分析该 vmcore 文件就可以诊断出内核崩溃的原因，从而进行操作系统的代码改进。主要用于分析以下问题场景：

- Android 系统内存泄露导致的卡顿、ANR 重启问题。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

- Linux 内核软件 Panic Oops 崩溃问题、死锁问题。
- Linux 内核态 Memory overflow 内存溢出、OOM 内存分配失败问题。

能够有效提升内核 panic 问题分析效率，更多用法见《Crashdump 调试工具使用指南》。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

11 FAQ

11.1 Android13 内核编译环境依赖于 libelf-dev 库，编译之前请务必安装 libelf-dev 库。

```
btf.c:18:10: fatal error: 'gelf.h' file not found
#include <gelf.h>
         ^~~~~~~~
  CC      /disk4/jfzhen/A133_T13.0_MID/longan/out/a133/kernel/build/tools/bpf/resolve_btfids/
    libbpf/staticobjs/btf_dump.o
  CC      /disk4/jfzhen/A133_T13.0_MID/longan/out/a133/kernel/build/tools/bpf/resolve_btfids/
    libbpf/staticobjs/ringbuf.o
  CC      /disk4/jfzhen/A133_T13.0_MID/longan/out/a133/kernel/build/tools/bpf/resolve_btfids/
    libbpf/staticobjs/strset.o
  CC      /disk4/jfzhen/A133_T13.0_MID/longan/out/a133/kernel/build/tools/bpf/resolve_btfids/
    libbpf/staticobjs/linker.o
1 error generated.
make[5]: *** [/disk4/jfzhen/A133_T13.0_MID/longan/out/a133/kernel/build/tools/bpf/
    resolve_btfids/libbpf/staticobjs/libbpf.o] Error 1
make[5]: *** Waiting for unfinished jobs....
make[3]: *** [/disk4/jfzhen/A133_T13.0_MID/longan/out/a133/kernel/build/tools/bpf/
    resolve_btfids//resolve_btfids-in.o] Error 2
make[3]: *** Waiting for unfinished jobs....
linker.c:17:10: fatal error: 'libelf.h' file not found
#include <libelf.h>
         ^~~~~~~~~~
1 error generated.
```

使用如下命令安装 libelf-dev 库:

```
apt install libelf-dev
```

11.2 Android13 编译过程中若突然退出，未编译完成再次编译，可能会出现无法继续编译的问题。

当编译过程出现如下异常打印:

```
20:12:23 Waiting up to 10s to lock out/.lock to ensure no other android build process is
    running in the same output directory
20:12:33 Tried to lock out/.lock, but timed out until 10s. Make sure no other android build
    process is using it
```

解决方法：删掉 out/.lock

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
android$ cd out/
android/out$ rm ./lock
```

广州星翼电子科技有限公司zdyzlinux

版权所有 © 珠海全志科技股份有限公司。保留一切权利

著作权声明

版权所有 ©2025 珠海全志科技股份有限公司。保留一切权利。

本文档及内容受著作权法保护，其著作权由珠海全志科技股份有限公司（“全志”）拥有并保留一切权利。

本文档是全志的原创作品和版权财产，未经全志书面许可，任何单位和个人不得擅自摘抄、复制、修改、发表或传播本文档内容的部分或全部，且不得以任何形式传播。

商标声明

ALLWINNER、ALLWINNER 全志科技、全志科技、ALLWINNER（不完全列举）均为珠海全志科技股份有限公司的商标或者注册商标。在本文档描述的产品中出现的其它商标，产品名称，和服务名称，均由其各自所有人拥有。

免责声明

您购买的产品、服务或特性应受您与珠海全志科技股份有限公司（“全志”）之间签署的商业合同和条款的约束。本文档中描述的全部或部分产品、服务或特性可能不在您所购买或使用的范围内。使用前请认真阅读合同条款和相关说明，并严格遵循本文档的使用说明。您将自行承担任何不当使用行为（包括但不限于如超压，超频，超温使用）造成的不利后果，全志概不负责。

本文档作为使用指导仅供参考。由于产品版本升级或其他原因，本文档内容有可能修改，如有变更，恕不另行通知。全志尽全力在本文档中提供准确的信息，但并不确保内容完全没有错误，因使用本文档而发生损害（包括但不限于间接的、偶然的、特殊的损失）或发生侵犯第三方权利事件，全志概不负责。本文档中的所有陈述、信息和建议并不构成任何明示或暗示的保证或承诺。

本文档未以明示或暗示或其他方式授予全志的任何专利或知识产权。在您实施方案或使用产品的过程中，可能需要获得第三方的权利许可。请您自行向第三方权利人获取相关的许可。全志不承担也不代为支付任何关于获取第三方许可的许可费或版税（专利税）。全志不对您所使用的第三方许可技术做出任何保证、赔偿或承担其他义务。